Технический аудит проекта tg_dorixona

Telegram-бот для анализа безопасности лекарств

---

1. Executive Summary

Проект: Telegram-бот `tg_dorixona` — система анализа безопасности сочетания лекарств для детей.

Статус: Проект находится в состоянии **рабочего MVP**, но содержит **критические архитектурные проблемы**, которые блокируют нормальный запуск и могут привести к потере данных.

Возможность восстановления: Да, проект можно восстановить без полного переписывания. Требуется **целевой рефакторинг** 3–5 ключевых файлов.

Главные выводы:

1. **Критический блокер:** Несоответствие между структурой БД в коде (`FamilyCard` без `childName`) и реальной БД на диске (содержит `childName`). Это приводит к потере данных при создании новых карточек.
2. **Проблема TypeScript конфигурации:** `tsconfig.json` использует `module: "Node16"` с `moduleResolution: "node16"` , что может вызвать проблемы с импортами и требует проверки на Node.js v22.
3. **Уязвимость хранилища:** JSON-файл `data/db.json` хранит все данные без шифрования, без версионирования и без резервных копий. Любая ошибка записи уничтожает данные.
4. **Отсутствие обработки ошибок в критичных местах:** Функции `createFamilyCard` , `addReminder` , `trackEvent` выполняют запись в БД, но не имеют транзакционности. Сбой на середине операции оставляет БД в несогласованном состоянии.
5. **Проблема инициализации:** Переменная `botUsername` имеет fallback `"your_bot_username"` , что может привести к некорректным ссылкам на бота.
6. **Потенциальная проблема с памятью:** `draftMap` и `stepMap` хранят состояние пользователей в памяти без очистки. При долгой работе бота это может привести к утечке памяти.

---

2. Project Snapshot

Параметр	Значение
Назначение проекта	Telegram-бот для анализа безопасности сочетания лекарств у детей (MVP)
Предполагаемый runtime	Node.js >= 18 (текущая версия: v22.13.0)
Язык	TypeScript 6.0.2
Главные фреймворки	Telegraf 4.16.3 (Telegram API), node-cron 4.2.1 (scheduler)
Дополнительные библиотеки	dayjs, dotenv, uuid
Менеджер пакетов	npm (lockfileVersion 3)
Модель модулей	ESM (Node16) с CommonJS совместимостью через `esModuleInterop`
Главная точка входа	`src/index.ts` → компилируется в `dist/index.js`
Главные npm scripts	`dev` (tsx src/index.ts), `build` (tsc), `start` (node dist/index.js)
Хранилище данных	JSON-файл `data/db.json` (нет БД)
Конфигурация	`.env` (BOT_TOKEN, BOT_USERNAME, APP_ENV)

---

3. Directory Map

Plain Text

```
/mnt/desktop/0_tg_dorixona/
├── src/
│   ├── index.ts                    # ГЛАВНЫЙ ENTRY POINT — весь бот, handlers,
│   ├── core/
│   │   ├── analysisEngine.ts       # Rule-based анализ лекарств (10 правил)
│   │   ├── medicationCatalog.ts    # Каталог 10 лекарств + парсер
│   │   ├── familyCardEngine.ts     # Создание/чтение карточек, история, deep l
│   │   ├── reminderEngine.ts       # Добавление/отправка напоминаний
│   │   ├── pharmacyEngine.ts       # Генерация ссылок на ArzonApteka
│   │   └── trackingEngine.ts       # Логирование событий в БД
│   ├── storage/
│   │   └── db.ts                   # JSON-хранилище, парсинг, нормализация, re
│   └── utils/
```

```
│       └── ids.ts                  # Генератор коротких ID (12 символов)
├── dist/                           # Скомпилированный JavaScript (пустой или у
├── data/
│   └── db.json                     # ЕДИНСТВЕННОЕ ХРАНИЛИЩЕ ДАННЫХ (16 KB, сод
├── node_modules/                   # npm зависимости
├── .env                            # Конфиг с реальным BOT_TOKEN (⚠️ SECURITY
├── .env.example                    # Шаблон конфига
├── package.json                    # Зависимости и scripts
├── tsconfig.json                   # TypeScript конфигурация
├── .gitignore                      # Игнорирует node_modules, dist, .env
├── README.md                       # Краткая документация
└── описание.txt                    # PowerShell скрипт-генератор проекта (исто
```

Назначение ключевых файлов:

- **src/index.ts** — Весь бот в одном файле: инициализация Telegraf, регистрация команд (`/start` , `/history`), обработчик текста (3-шаговый wizard), callback-действия (toggle purchase), cron-scheduler для напоминаний, graceful shutdown.
- **src/storage/db.ts** — Слой абстракции для JSON-хранилища. Читает `data/db.json` , парсит, нормализует структуру, восстанавливается при ошибках парсинга, хранит последнюю ошибку в памяти.
- **src/core/familyCardEngine.ts** — Управление карточками: создание (с проверкой ошибок БД), чтение по ID, получение истории пользователя (последние 10), генерация deep-link для шаринга.
- **src/core/analysisEngine.ts** — Правила анализа: 2+ антибиотика → "attention", 2+ антигистамина → "caution", парацетамол+ибупрофен → "caution", амоксициллин+амоксиклав → "attention", салбутамол+будесонид → "safe" с пояснением.
- **src/core/medicationCatalog.ts** — Жесткий каталог 10 лекарств с синонимами. Парсер ищет совпадения substring в обе стороны.
- **src/core/reminderEngine.ts** — CRUD для напоминаний: добавление, получение ожидающих (по времени), отметить как отправленное.
- **src/core/trackingEngine.ts** — Простой append-only логгер событий в БД (open_card, create_card, toggle_purchase_item).
- **src/utils/ids.ts** — Генератор ID: `crypto.randomUUID()` → удалить дефисы → первые 12 символов.

---

4. Launch Path

Цепочка инициализации от npm script до готовности бота:

Plain Text

```
npm run dev
  ↓
tsx src/index.ts (esbuild-based TypeScript runner)
  ↓
import "dotenv/config"
  → Читает .env, устанавливает process.env.BOT_TOKEN, BOT_USERNAME, APP_ENV
  ↓
Проверка: if (!token || token === "PASTE_YOUR_BOT_TOKEN_HERE") throw Error
  → Если токена нет → CRASH
  ↓
new Telegraf(token)
  → Инициализирует Telegram API клиент
  ↓
Регистрация handlers:
  - bot.start() → обработка /start и deep-links card_*
  - bot.command("history") → /history
  - bot.on("text") → 3-шаговый wizard (age → symptoms → medications)
  - bot.action(/toggle_/) → callback для чекбокса в purchase list
  ↓
cron.schedule("*/5 * * * *", async () => {...})
  → Запуск scheduler для напоминаний каждые 5 минут
  ↓
bot.launch()
  → Подключение к Telegram API, начало polling обновлений
  ↓
process.once("SIGINT"/"SIGTERM", () => bot.stop())
  → Graceful shutdown на Ctrl+C или SIGTERM
```

Поток обработки сообщения пользователя:

Plain Text

```
Пользователь отправляет текст
  ↓
bot.on("text", async (ctx) => {...})
  ↓
Определение текущего шага: stepMap.get(userId) || "age"
  ↓
Валидация input:
  - age: /^-?\d+$/ и 0 <= age <= 18
  - symptoms: length >= 3
  - medications: parseMedications() → должно вернуть массив
```

```
  ↓
Если валидация не пройдена → ctx.reply("...") и return
  ↓
Если medications:
  - analyzeMedications(meds) → AnalysisResult
  - createFamilyCard({userId, age, symptoms, medications, analysis}) → FamilyCa
  - addReminder(userId, cardId, Date.now() + 6h)
  - trackEvent(userId, "create_card", {cardId, meds})
  - ctx.reply(summaryText, inlineKeyboard)
  ↓
Очистка draft: draftMap.delete(userId), stepMap.delete(userId)
```

Поток напоминаний (каждые 5 минут):

Plain Text

```
cron.schedule("*/5 * * * *")
  ↓
getPendingReminders(Date.now())
  → Читает db.reminders, фильтрует !sent && runAt <= now
  ↓
Для каждого напоминания:
  - bot.telegram.sendMessage(userId, "Напоминание: ...")
  - markReminderSent(userId, cardId) → устанавливает sent = true в БД
  ↓
Если ошибка отправки → console.error(), но не падает
```

---

5. Environment & Config Audit

Переменная	Обязательна	Где читается	Значение по умолчанию	Риск
BOT_TOKEN	✅ ДА	`src/index.ts:12`	Нет (CRASH если отсутствует или `PASTE_YOUR_BOT_TOKEN_HERE`)	**CRITICAL** — бот не запустится
BOT_USERNAME	❌ НЕТ	`src/index.ts:13`	`"your_bot_username"`	**HIGH** — неправильные deep-links, шаринг сломан

APP_ENV	❌ НЕТ	Читается в `.env`, но **не используется в коде**	development	**LOW** — мертвый код

Проверка .env vs .env.example:

Файл	BOT_TOKEN	BOT_USERNAME	APP_ENV
.env	✅ Заполнен (реальный токен)	✅ `SalesDataUzBot`	✅ `development`
.env.example	❌ `PASTE_YOUR_BOT_TOKEN_HERE`	❌ `your_bot_username`	✅ `development`

Проблемы:

1. **Реальный BOT_TOKEN в .env** — Это публичный репозиторий на рабочем столе. Токен скомпрометирован. ⚠️ **SECURITY ISSUE**
2. **Несоответствие .env.example** — Если разработчик клонирует проект и скопирует `.env.example` в `.env`, бот не запустится с понятной ошибкой.
3. **APP_ENV не используется** — Переменная читается, но никогда не используется в коде. Это мертвый код.
4. **Отсутствие валидации env при старте** — Нет проверки на наличие `data/` директории, нет проверки на права доступа к `data/db.json`.

---

6. Dependency & Integration Audit

Зависимость	Версия	Где используется	Зачем нужна	Риск	Критичность
telegraf	4.16.3	`src/index.ts` (весь бот)	Telegram Bot API клиент	**MEDIUM** — если API изменится, может сломаться	**CRITICAL**

node-cron	4.2.1	`src/index.ts:355` (scheduler напоминаний)	Cron-like scheduler для Node.js	**LOW** — стабильная библиотека	**HIGH**
dotenv	17.3.1	`src/index.ts:1` (import "dotenv/config")	Загрузка .env в process.env	**LOW**	**CRITICAL**
dayjs	1.11.20	**НЕ ИСПОЛЬЗУЕТСЯ** ❌	Парсинг дат (импортирован, но не применяется)	**LOW**	**NONE**
uuid	13.0.0	**НЕ ИСПОЛЬЗУЕТСЯ** ❌	Генерация UUID (есть `src/utils/ids.ts`, но использует `crypto.randomUUID()`)	**LOW**	**NONE**
@types/node	25.5.0	tsconfig.json: `"types": ["node"]`	TypeScript типы для Node.js API	**LOW**	**HIGH**
typescript	6.0.2	`npm run build` (tsc)	TypeScript компилятор	**MEDIUM** — версия 6.0.2 очень новая, может быть нестабильной	**HIGH**
tsx	4.21.0	`npm run dev` (tsx src/index.ts)	TypeScript executor для Node.js (esbuild-based)	**LOW**	**HIGH**

Внешние интеграции:

Интеграция	Тип	Где используется	Риск	Статус
Telegram API	HTTP REST	`src/index.ts` (Telegraf)	**MEDIUM** — зависит от доступности Telegram	✅ Работает
ArzonApteka	HTTP URL (search link)	`src/core/pharmacyEngine.ts`	**LOW** — это просто ссылка, не API	✅ Работает
JSON DB (data/db.json)	Файловая система	`src/storage/db.ts`	**CRITICAL** — нет резервных копий, нет транзакций	⚠️ Уязвимо

---

7. Logic & Architecture Findings

Сильные стороны:

1. **Четкое разделение ответственности** — Каждый модуль в `src/core/` отвечает за одну функцию (анализ, каталог, карточки, напоминания, трекинг).
2. **Graceful error handling в некоторых местах** — `createFamilyCard` проверяет ошибки БД и уведомляет пользователя. Cron-scheduler не падает при ошибке отправки напоминания.
3. **Recovery механизм в DB layer** — `readDb()` может восстановиться из поврежденного JSON, перезаписав БД пустой структурой.
4. **Правила анализа логичные и понятные** — Правила в `analysisEngine.ts` основаны на реальных фармацевтических взаимодействиях.
5. **Поддержка deep-linking** — Бот может открывать сохраненные карточки через `/start card_<id>`.

Слабые места:

1. **Структурное несоответствие БД** — `FamilyCard` тип не содержит `childName`, но реальная БД содержит это поле. Новые карточки будут созданы без `childName`, что приведет к потере данных при миграции.

   Гипотеза: Проект был переработан, но БД не была мигрирована.

2. **Отсутствие транзакционности** — Функции `createFamilyCard`, `addReminder`, `trackEvent` выполняют несколько операций с БД, но нет гарантии атомарности. Если процесс упадет между `readDb()` и `writeDb()`, БД будет в несогласованном состоянии.
3. **Утечка памяти в draftMap и stepMap** — Эти Map хранят состояние пользователей, но не очищаются автоматически. Если пользователь не завершит wizard, его запись останется в памяти навсегда.

   Пример проблемы: После 1000 пользователей, которые начали, но не завершили wizard, `draftMap` будет содержать 1000 записей, потребляя память.
4. **Отсутствие валидации при чтении из БД** — `getFamilyCard()`, `getUserHistory()` не проверяют, что данные имеют ожидаемую структуру. Если БД содержит поврежденные записи, это может привести к runtime ошибкам.
5. **Парсер лекарств слишком либеральный** — `parseMedications()` использует substring matching в обе стороны, что может привести к ложным совпадениям:
   - Ввод: "цет" → может совпасть с "цетиризин" ✅
   - Ввод: "иб" → может совпасть с "ибупрофен" ✅
   - Ввод: "ам" → может совпасть с "амоксициллин" И "амоксиклав" (оба добавятся) ⚠️
6. **Отсутствие логирования** — Нет структурированного логирования. Только `console.error()` для отладки. Невозможно отследить, что произошло в production.
7. **Жесткий каталог лекарств** — Только 10 лекарств в `MEDICATIONS`. Если пользователь введет неизвестное лекарство, парсер его не найдет и вернет пустой массив.
8. **Отсутствие rate-limiting** — Нет защиты от spam. Пользователь может отправлять сообщения бесконечно, и бот будет обрабатывать все.

Неоднозначные места:

1. **Fallback для BOT_USERNAME** — Если `BOT_USERNAME` не установлена, используется `"your_bot_username"`, что создает некорректные deep-links. Это может быть намеренно (для разработки), но в production это проблема.
2. **Обработка ошибок в toggle_action** — Если `getFamilyCard()` вернет `undefined`, бот отправляет `answerCbQuery("Карточка не найдена")`, но не логирует это. Это может быть признаком того, что карточка была удалена или БД повреждена.
3. **Отсутствие проверки прав доступа** — Любой пользователь может открыть любую карточку по deep-link, если знает ID. Нет проверки, что `userId` в карточке совпадает с `ctx.from.id`.

Гипотеза: Это может быть намеренно (для шаринга), но это потенциальная уязвимость приватности.

---

8. Startup Blockers

Блокер 1: Отсутствие BOT_TOKEN

Параметр	Значение
Блокер	`if (!token || token === "PASTE_YOUR_BOT_TOKEN_HERE") throw new Error("Укажи BOT_TOKEN в .env")`
Где найден	`src/index.ts:15–17`
Почему ломает	Бот не может подключиться к Telegram API без валидного токена. Процесс упадет с ошибкой.
Вероятность	**VERY HIGH** — если разработчик скопирует `.env.example` в `.env` и забудет заполнить токен
Что нужно проверить	Наличие `BOT_TOKEN` в `.env`, его формат (должен быть `<number>:<string>`)

Блокер 2: Отсутствие data/ директории

Параметр	Значение
Блокер	`readDb()` → `ensureDb()` → `fs.mkdirSync(path.dirname(dbPath), { recursive: true })`
Где найден	`src/storage/db.ts:34–39`
Почему ломает	Если директория `data/` не существует, она будет создана автоматически. Но если процесс не имеет прав на запись в текущую директорию, это вызовет ошибку.
Вероятность	**MEDIUM** — если проект запущен из директории, где нет прав на запись

Что нужно проверить	Права доступа на запись в `process.cwd()`, наличие `data/` директории

Блокер 3: TypeScript конфигурация несовместима с Node.js v22

Параметр	Значение
Блокер	`tsconfig.json`: `"module": "Node16"`, `"moduleResolution": "node16"`
Где найден	`tsconfig.json:4–5`
Почему ломает	Node16 ESM resolution может быть несовместима с Node.js v22. Это может привести к ошибкам импорта или неправильному разрешению модулей.
Вероятность	**LOW** — Node16 обычно работает с v22, но требует проверки
Что нужно проверить	Запустить `npm run build` и проверить, что компиляция успешна. Запустить `npm run dev` и проверить, что нет ошибок импорта.

Блокер 4: Структурное несоответствие БД

Параметр	Значение
Блокер	`FamilyCard` тип не содержит `childName`, но реальная БД содержит это поле
Где найден	`src/core/familyCardEngine.ts:5–13` vs `data/db.json`
Почему ломает	Новые карточки будут созданы без `childName`, что приведет к потере данных. При чтении старых карточек `childName` будет проигнорирован.
Вероятность	**VERY HIGH** — это уже происходит в текущей БД
Что нужно проверить	Сравнить структуру `FamilyCard` с реальными записями в `data/db.json`. Запустить

	миграцию БД.

Блокер 5: Отсутствие обработки ошибок при инициализации

Параметр	Значение
Блокер	`bot.launch()` не имеет обработчика ошибок
Где найден	`src/index.ts:371–372`
Почему ломает	Если Telegram API недоступна или токен невалиден, `bot.launch()` вернет отклоненный Promise, и процесс упадет с необработанной ошибкой.
Вероятность	**MEDIUM** — если Telegram API временно недоступна
Что нужно проверить	Добавить `.catch()` обработчик к `bot.launch()`

---

9. Functional Risk Map

Зона	Риск	Симптом	Вероятная причина	Приоритет
Запуск бота	Процесс падает с ошибкой	"Укажи BOT_TOKEN в .env"	Отсутствует или неправильный `BOT_TOKEN`	**P0**
Создание карточки	Данные теряются	Новая карточка не содержит `childName`, старые карточки содержат	Структурное несоответствие БД	**P0**
Чтение БД	Процесс падает	JSON parse error	Поврежденный `data/db.json`	**P1**

Напоминания	Напоминание не отправляется	Пользователь не получает сообщение	Ошибка в `bot.telegram.sendMessage()` или неправильное время `runAt`	**P1**
Deep-link шаринга	Ссылка некорректна	Ссылка содержит `your_bot_username` вместо реального имени	`BOT_USERNAME` не установлена или установлена неправильно	**P1**
Парсинг лекарств	Лекарство не распознается	Пользователь вводит лекарство, но парсер возвращает пустой массив	Лекарство отсутствует в каталоге или парсер не может найти совпадение	**P2**
Утечка памяти	Бот потребляет все больше памяти	Процесс медленнее, затем падает	`draftMap` и `stepMap` не очищаются	**P2**
Spam/DoS	Бот перегружен	Медленный ответ на сообщения	Отсутствует rate-limiting	**P2**
Приватность	Уязвимость	Пользователь может открыть чужую карточку	Отсутствует проверка прав доступа при открытии карточки	**P1**

---

10. Critical Files for Recovery

Список файлов, которые нужно изучать и чинить в первую очередь (по приоритету):

P0 (Блокирует запуск):

1. **src/index.ts** — Главный entry point. Требуется:
   - Добавить обработчик ошибок к `bot.launch()`
   - Добавить проверку `BOT_USERNAME` при инициализации

- Добавить логирование инициализации

2. **src/core/familyCardEngine.ts** — Структурное несоответствие БД. Требуется:
   - Обновить тип `FamilyCard` для включения `childName` (или удалить из БД)
   - Добавить миграцию БД для старых записей
   - Добавить валидацию при чтении карточек
3. **src/storage/db.ts** — Уязвимое хранилище. Требуется:
   - Добавить резервное копирование перед записью
   - Добавить валидацию структуры при чтении
   - Добавить логирование всех операций

P1 (Запускается, но ломает функционал):

1. **.env** — Содержит реальный токен. Требуется:
   - Ротация токена (текущий скомпрометирован)
   - Переместить в `.env.local` или другое безопасное место
   - Обновить `.gitignore`
2. **src/core/reminderEngine.ts** — Напоминания могут не отправляться. Требуется:
   - Добавить обработку ошибок при отправке
   - Добавить логирование
3. **src/index.ts** (cron scheduler) — Требуется:
   - Добавить обработку ошибок при отправке напоминаний
   - Добавить логирование успешных отправок

P2 (Улучшения):

1. **src/index.ts** (draftMap/stepMap) — Утечка памяти. Требуется:
   - Добавить автоматическую очистку старых записей (например, через 1 час)
   - Или использовать WeakMap
2. **src/core/medicationCatalog.ts** — Жесткий каталог. Требуется:
   - Расширить каталог или сделать его загружаемым из конфига
   - Улучшить парсер для избежания ложных совпадений
3. **src/index.ts** — Отсутствие rate-limiting. Требуется:
   - Добавить простой rate-limiter (например, 10 сообщений в минуту на пользователя)

---

11. Safe Recovery Plan

Пошаговый план восстановления без переписывания архитектуры:

Фаза 1: Критические блокеры (P0) — 1–2 часа

Шаг 1.1: Проверить и зафиксировать структуру БД

- Прочитать текущую `data/db.json`
- Определить, какие поля есть в реальных карточках
- Обновить тип `FamilyCard` в `src/core/familyCardEngine.ts` для соответствия реальной структуре
- Или удалить `childName` из всех существующих карточек в БД

Шаг 1.2: Добавить обработку ошибок при инициализации бота

- Обновить `src/index.ts:371–376` для добавления `.catch()` обработчика к `bot.launch()`
- Добавить логирование: "Bot started" или "Bot failed to start: ..."

Шаг 1.3: Валидировать конфигурацию при старте

- Добавить проверку: `if (!botUsername || botUsername === "your_bot_username") console.warn("BOT_USERNAME не установлена")`
- Добавить проверку: `if (!fs.existsSync("data")) fs.mkdirSync("data", { recursive: true })`

Шаг 1.4: Протестировать компиляцию и запуск

- `npm run build` — должно завершиться без ошибок
- `npm run dev` — должно вывести "Bot started"

Фаза 2: Функциональные проблемы (P1) — 2–3 часа

Шаг 2.1: Ротация BOT_TOKEN

- Создать новый токен в Telegram BotFather
- Обновить `.env` с новым токеном
- Удалить старый токен из истории

Шаг 2.2: Добавить обработку ошибок в критичных местах

- `src/core/reminderEngine.ts` — добавить try-catch в `markReminderSent()`
- `src/index.ts` (cron scheduler) — улучшить обработку ошибок при отправке напоминаний
- `src/core/familyCardEngine.ts` — добавить валидацию при чтении карточек

Шаг 2.3: Добавить резервное копирование БД

- Перед `writeDb()` в `src/storage/db.ts` создавать резервную копию: `data/db.json.backup`
- Добавить функцию восстановления из резервной копии

Шаг 2.4: Добавить проверку прав доступа при открытии карточки

- В `src/index.ts` (bot.start handler) добавить проверку: `if (card.userId !== userId) { await ctx.reply("Нет доступа"); return; }`

Фаза 3: Оптимизации (P2) — 1–2 часа

Шаг 3.1: Добавить автоматическую очистку draftMap/stepMap

- Добавить функцию `cleanupOldDrafts()` которая удаляет записи старше 1 часа
- Запустить её через `setInterval()` каждые 10 минут

Шаг 3.2: Улучшить парсер лекарств

- Добавить более точное совпадение (не только substring)
- Например, использовать Levenshtein distance для нечеткого поиска

Шаг 3.3: Добавить простой rate-limiter

- Создать Map `userMessageCounts` для отслеживания количества сообщений
- Если пользователь отправил > 10 сообщений за 1 минуту, отправить "Пожалуйста, не спешите"

Фаза 4: Тестирование и документирование — 1 час

Шаг 4.1: Протестировать основной flow

- Отправить `/start` → введить возраст → симптомы → лекарства → проверить создание карточки
- Отправить `/history` → проверить, что карточка в истории
- Открыть deep-link → проверить, что карточка открывается

Шаг 4.2: Протестировать обработку ошибок

- Отправить невалидный возраст → проверить ошибку
- Отправить неизвестное лекарство → проверить ошибку
- Отключить Telegram API → проверить, что бот не падает

Шаг 4.3: Обновить документацию

- Обновить `README.md` с инструкциями по ротации токена
- Добавить раздел "Troubleshooting"

- Документировать структуру БД

---

12. Handoff for Coding Agent

Техническое задание для coding agent

Цель: Восстановить проект `tg_dorixona` до состояния, когда он может надежно запуститься и работать без потери данных.

Контекст:

- Проект находится в `/mnt/desktop/0_tg_dorixona/`
- Это Telegram-бот для анализа безопасности лекарств
- Стек: Node.js v22, TypeScript 6.0.2, Telegraf 4.16.3, JSON-хранилище
- Главный entry point: `src/index.ts`

Задачи в порядке приоритета:

TASK 1: Исправить структурное несоответствие БД (P0)

Описание: Тип `FamilyCard` в коде не содержит поле `childName`, но реальная БД (`data/db.json`) содержит это поле. Это приводит к потере данных при создании новых карточек.

Что нужно сделать:

1. Прочитать текущую `data/db.json` и определить точную структуру карточек
2. Обновить тип `FamilyCard` в `src/core/familyCardEngine.ts` для соответствия реальной структуре (добавить `childName?: string`)
3. Обновить функцию `createFamilyCard()` для сохранения `childName` (если передано)
4. Добавить миграцию: функция `migrateDb()` которая обновляет старые карточки без `id` и `createdAt`
5. Протестировать: создать новую карточку и проверить, что она содержит все поля

Файлы для изменения:

- `src/core/familyCardEngine.ts` — обновить тип и функции
- `src/storage/db.ts` — добавить функцию миграции
- `src/index.ts` — вызвать миграцию при старте (один раз)

Критерии успеха:

- Новые карточки содержат все поля (включая `childName`)

- Старые карточки остаются неповрежденными
- Нет потери данных

TASK 2: Добавить обработку ошибок при инициализации бота (P0)

Описание: `bot.launch()` не имеет обработчика ошибок. Если Telegram API недоступна, процесс упадет с необработанной ошибкой.

Что нужно сделать:

1. Добавить `.catch()` обработчик к `bot.launch()` в `src/index.ts:371–372`
2. При ошибке: логировать ошибку и завершить процесс с кодом 1
3. Добавить логирование успешного старта: "Bot started successfully"
4. Добавить проверку `BOT_USERNAME` при старте: если не установлена, логировать warning

Файлы для изменения:

- `src/index.ts` — обновить инициализацию бота

Критерии успеха:

- Если Telegram API недоступна, бот логирует ошибку и завершается
- Если все ОК, бот логирует "Bot started successfully"
- Нет необработанных ошибок

TASK 3: Добавить валидацию конфигурации при старте (P0)

Описание: Отсутствует проверка, что `data/` директория существует и имеет права на запись.

Что нужно сделать:

1. Добавить функцию `validateConfig()` в `src/index.ts`
2. Проверить: `process.env.BOT_TOKEN` установлен и не равен "PASTE_YOUR_BOT_TOKEN_HERE"
3. Проверить: `process.env.BOT_USERNAME` установлена (если нет, логировать warning)
4. Проверить: `data/` директория существует, если нет — создать
5. Проверить: права на запись в `data/` (попытаться создать временный файл)
6. Вызвать `validateConfig()` при старте, перед `bot.launch()`

Файлы для изменения:

- `src/index.ts` — добавить функцию валидации

Критерии успеха:

- Все проверки выполняются при старте
- Если проверка не пройдена, бот логирует ошибку и завершается
- Если все ОК, бот продолжает инициализацию

TASK 4: Добавить резервное копирование БД (P1)

Описание: Нет резервных копий БД. Если `writeDb()` упадет на середине, БД будет повреждена.

Что нужно сделать:

1. Добавить функцию `backupDb()` в `src/storage/db.ts`
2. Перед каждой записью в `writeDb()` создавать резервную копию: `data/db.json.backup`
3. Хранить максимум 3 последних резервных копии: `data/db.json.backup.1`, `db.json.backup.2`, `db.json.backup.3`
4. Добавить функцию `restoreDbFromBackup()` для восстановления из резервной копии
5. Логировать все операции резервного копирования

Файлы для изменения:

- `src/storage/db.ts` — добавить функции резервного копирования

Критерии успеха:

- Перед каждой записью создается резервная копия
- Максимум 3 резервных копии хранятся
- Функция восстановления работает

TASK 5: Добавить проверку прав доступа при открытии карточки (P1)

Описание: Любой пользователь может открыть любую карточку по deep-link, если знает ID. Нет проверки, что `userId` в карточке совпадает с `ctx.from.id`.

Что нужно сделать:

1. В `src/index.ts` (bot.start handler, строка 112–119) добавить проверку прав доступа
2. Если `card.userId !== userId`, отправить "Нет доступа к этой карточке"

3. Логировать попытку несанкционированного доступа

Файлы для изменения:

- `src/index.ts` — добавить проверку в bot.start handler

Критерии успеха:

- Пользователь не может открыть чужую карточку
- Попытка несанкционированного доступа логируется

TASK 6: Добавить автоматическую очистку draftMap/stepMap (P2)

Описание: `draftMap` и `stepMap` не очищаются, что приводит к утечке памяти.

Что нужно сделать:

1. Добавить функцию `cleanupOldDrafts()` в `src/index.ts`
2. Функция должна удалять записи старше 1 часа из `draftMap` и `stepMap`
3. Запустить функцию через `setInterval()` каждые 10 минут
4. Логировать количество удаленных записей

Файлы для изменения:

- `src/index.ts` — добавить функцию очистки

Критерии успеха:

- Старые записи удаляются автоматически
- Память не растет бесконечно
- Логирование работает

TASK 7: Добавить простой rate-limiter (P2)

Описание: Отсутствует защита от spam. Пользователь может отправлять сообщения бесконечно.

Что нужно сделать:

1. Добавить Map `userMessageCounts` для отслеживания количества сообщений
2. В `bot.on("text")` handler добавить проверку: если пользователь отправил > 10 сообщений за 1 минуту, отправить "Пожалуйста, не спешите"
3. Очищать счетчик каждую минуту
4. Логировать попытки spam

Файлы для изменения:

- `src/index.ts` — добавить rate-limiter

Критерии успеха:

- Spam блокируется
- Обычные пользователи не затронуты
- Логирование работает

TASK 8: Протестировать и задокументировать (P2)

Описание: Нужно убедиться, что все исправления работают и задокументировать процесс.

Что нужно сделать:

1. Запустить `npm run build` — должно завершиться без ошибок
2. Запустить `npm run dev` — должно вывести "Bot started successfully"
3. Протестировать основной flow: `/start` → возраст → симптомы → лекарства → карточка
4. Протестировать `/history` — должна показать созданную карточку
5. Протестировать deep-link — должна открыться карточка
6. Протестировать обработку ошибок: невалидный возраст, неизвестное лекарство
7. Обновить `README.md` с инструкциями по запуску и troubleshooting
8. Добавить раздел "Architecture" с описанием структуры проекта

Файлы для изменения:

- `README.md` — обновить документацию

Критерии успеха:

- Все тесты пройдены
- Документация актуальна
- Нет необработанных ошибок

Порядок выполнения:

1. **TASK 1** → TASK 2 → TASK 3 (критические блокеры, должны быть выполнены первыми)
2. **TASK 4** → TASK 5 (функциональные проблемы)
3. **TASK 6** → TASK 7 (оптимизации)

4. **TASK 8** (финальное тестирование)

Примечания для coding agent:

- Не переписывайте архитектуру. Работайте в рамках существующей структуры.
- Добавляйте логирование для всех критичных операций.
- Тестируйте каждое изменение перед переходом к следующему.
- Если столкнетесь с неожиданной проблемой, логируйте её и сообщите в отчете.
- Сохраняйте обратную совместимость с существующей БД.

Дополнительная информация

Файлы, которые НЕ нужно трогать (если не требуется):

- `src/core/medicationCatalog.ts` — каталог лекарств (P2 улучшение)
- `src/core/analysisEngine.ts` — правила анализа (работают корректно)
- `src/core/pharmacyEngine.ts` — ссылки на аптеку (работают корректно)
- `src/core/trackingEngine.ts` — трекинг событий (работает корректно)
- `src/utils/ids.ts` — генератор ID (работает корректно)
- `tsconfig.json` — конфигурация TypeScript (работает корректно)
- `.gitignore` — игнор файлов (работает корректно)

Файлы, которые нужно обновить:

- `src/index.ts` — главный файл, требует множества исправлений
- `src/core/familyCardEngine.ts` — структурное несоответствие БД
- `src/storage/db.ts` — добавить резервное копирование и валидацию
- `.env` — ротация токена (security issue)
- `README.md` — обновить документацию

Гипотезы, требующие проверки:

1. **Структурное несоответствие БД** — Проект был переработан, но БД не была мигрирована. Нужно проверить, когда был создан `data/db.json` и когда был обновлен код.

2. **TypeScript v6.0.2 нестабильна** — Версия 6.0.2 очень новая и может содержать баги. Нужно проверить, работает ли компиляция без ошибок.
3. **Node16 ESM resolution** — Может быть несовместима с Node.js v22. Нужно протестировать запуск бота.
4. **Реальный BOT_TOKEN в репозитории** — Токен скомпрометирован. Нужно ротировать его немедленно.

Дата аудита: 30 марта 2026 г.

Версия отчета: 1.0

Статус: Готово к передаче coding agent